

SWEEP rule on the KD-Toric code

February 10, 2026

1 SWEEP Rule Decoder on the Toric

The decoder works as follows: each vertex v looks for a suggestion $\hat{v}$ on $\uparrow(v) \cap lk_2$ that matches σ_v^+ . If there are more than one, then pick the one which has the minimal weight.

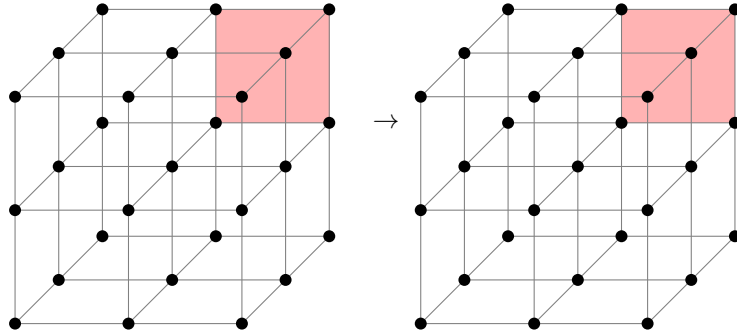


Figure 1: SWEEP rule

2 Span Expansion

Define the potentials functions $L_i : \mathbb{R}^d \rightarrow \mathbb{R}$ as follow¹:

$$L_i(\mathbf{x}) = \begin{cases} \mathbf{x}_i & i < d + 1 \\ -\langle \mathbf{1}, \mathbf{x} \rangle & i = d + 1 \end{cases}$$

The **Size** of a subset S is defined as:

$$\text{Size}(S) = \sum_i \max_{\mathbf{x} \in S} L_i(\mathbf{x})$$

Numerically, we show that the span property holds for the checks. Namely, let e be an unsatisfied check at time t , and let H_e be the unsatisfied checks at time $t - 1$. Then there is a set $H'_e \subset H_e$ such that:

1. For any $i < d + 1$ there exists $e' \in H'_e$ such $L_i(e') \geq L_i(e)$.
2. There exists $e' \in H'_e$ such that $L_{d+1}(e') \leq L_{d+1}(e) + 1$.

Proof. Consider a vertex v which at time t adjoins to an unsatisfied edge e , let u, w be vertices which have a positive faces that adjoin to e .

¹Notice that we use opposite signs relative to Berman and Peter Gacs, yet we agree with Perskil.

1. If $\hat{u}|_e \neq \emptyset$, then the parallel edge to e going out from u belongs to H_e . Denote it by e_u , taking e_u satisfies at least two (three in 4D) of the first d inequalities and the last $d + 1$ th inequality. Denote by $x \in [d]$ the index of the inequality which isn't satisfied by e_u . So it remains to prove the existence of $e' \in H_e$ for which $L_x(e') \geq L_x(e)$.

Consider the assignment of bits at time $t - 1$ over the complex, and consider the graph $G = (\mathcal{F}, E)$ induced by associating vertices with any face that is set to 1 by the assignment. Two vertices are connected if their origin faces share an edge. Let $K \subset \mathcal{F}$ be a connected component of faces that contains the faces supported on u .

We say that K is d far from u if:

$$\max_{f \in K} |f_x - u_x| \leq d$$

Now, we will prove by induction on d that there is a face $f \in K$ with $f_x \geq v_x$ such one of its edge is not satisfied.

- (a) **Base.** The edge from v which point at the $\hat{z}$ direction must not be satisfied.

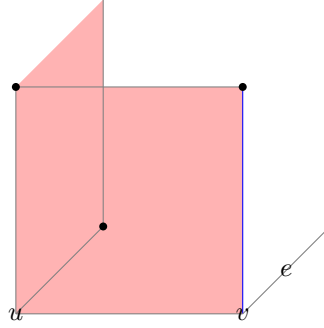


Figure 2: The induction base: In red, we have the faces on the support of u . The blue edge has the same x -coordinate as e . Any attempt to zero out the syndrome on the edge results in adding a face that is at least 2-far from u . Any configuration which is at most 1-far from u has to have that local view, yet might be connected to more faces which don't adjoin the blue edge.

- (b) **Assumption.** Assume that for any K which is at most $d - 1$ far from u there is a face, with adjoins edge that is both has a x -coordinate that is greater than u_x and is not satisfied.
- (c) **Induction Step.** Consider a connected component K that is d -far from u . Notice that if by substitute a co-boundary (stabilizer) from K we get K' that is at most d' far from u for some $d' < d$ then we done. That because: $\partial_2 K' = \partial_2(K + z) = \partial_2 K$ for some $z \in \ker \partial_2$.

On the other hand, [\[COMMENT\]](#) Finish that.

2. Else, if for any $u \in \downarrow(u) \cap lk_1$ it holds that $\hat{u}|_e = \emptyset$, then it must be that $e \in H_e$ because $\hat{v}$ can only decrease the syndrome on v . Moreover, it follows that $\hat{v} = \emptyset$ (otherwise, the suggestion would zero out the syndrome on $\uparrow(v) \cap lk_2$). Thus, $\sigma_v \not\subset \uparrow(v) \cap lk_2 \Rightarrow$ there is also an edge $e' \in H_e$ where $e' \in \downarrow(v)$. In that case, take $e \in H_e$ as the edge satisfies the first d equalities and e' for satisfying the $d + 1$ th inequality.

□

3 Investigation

[COMMENT] To do: understands how the investigation should look like. Plot an example to such tree. The model goes as follow, at each even iteration we toss a noise, then in each odd iteration we perform a single SWEEP rule iteration. That allows to track over the history of the errors.

We say that a check $e \in \text{Error}$, if there is a face f that adjoint to e and belongs to the Error.

Definition 3.1 (Investigation). For any unsatisfied check e at time t , we define V (the set of checks which participated in 'misleading' of e), and two sets of edges on V , Arrows and Forks via the following algorithm:

```

1 let  $V = \{e\}$ , Arrows =  $\emptyset$ ,
  Forks =  $\emptyset$ 
2 let  $Q \leftarrow \text{queue } \{e\}$ 
3 while  $Q$  is not empty do
4    $e' \leftarrow \text{dequeue } Q$ 
5   if  $e' \notin \text{Error}$  then
6      $e_1, \dots, e_l \leftarrow \text{Excuse}(e')$ 
7     Insert  $\{e_i, e'\}$  to
      Arrows
8     Insert  $\{e_i, e_j\}$  to
      Forks
9   end
10 end

```

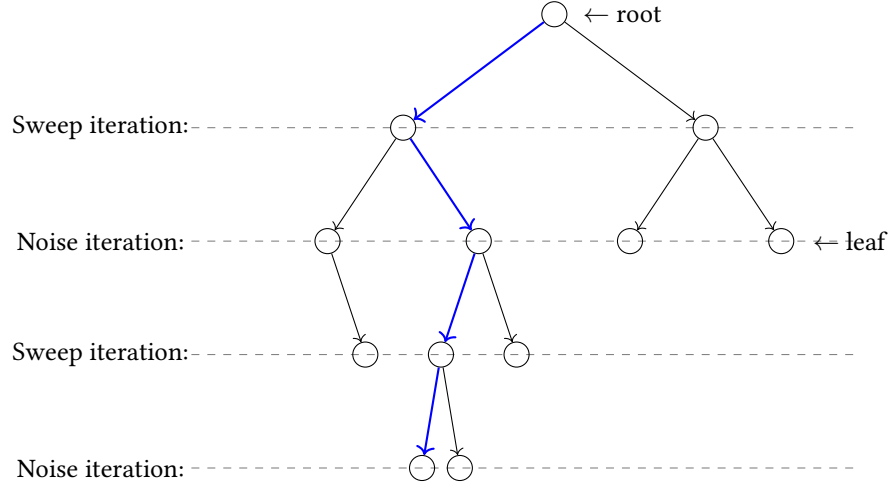


Figure 3: blabla

Definition 3.2 (History, slice.). G_u is the subgraph of $\langle V, \text{Arrows} \rangle$ induced by $\{v \in V : \text{Time}(v) \leq \text{Time}(u)\}$. **History**(u) is the connecting connected component of G_u containing u . A slice is an equivalence class of the relation $u \equiv v$ iff **History**(u) = **History**(v). Notice that if u and v are in the same slice K , then $\text{Time}(u) = \text{Time}(v)$. Thus we can write **Time**(K) and **History**(K) without ambiguity.

Claim 3.1. If for two different slices K_1 and K_2 we have that **Time**(K_1) = **Time**(K_2), then **History**(K_1) and **History**(K_2) are disjoint.

Proof. Directly from definitions. □

Definition 3.3 (The graph of slice). For a slice K we define the graph $G_K = \langle V_K, E_K \rangle$ as follows:

1. $V_K = \{\text{slice } R : R \subset \text{History}(K) \text{ and } \text{Time}(R) = \text{Time}(K) - 1\}$
2. $\{R, S\} \in E_K$ iff $R, S \in V_K$ and for some $\{v, w\} \in \text{Forks}$ we have $v \in R$ and $w \in S$.

Claim 3.2. G_K is connected.

Proof. Assume R and S are from V_K and that $v \in R, w \in S$. Since $v, w \in \text{History}(K)$, there exists a path from v to w in **History**(K) consisting of arrows. Select this path in such a way that it contains the minimal number of points x such that $\text{Time}(x) = \text{Time}(K)$. By induction on the number k of such points we prove that R and S are in the same connected component of G_K .

1. Base. $k = 0$, In this case, the path from v to w lies within $G_v (= G_w)$. Hence **History**(v) = **History**(w) which means that the slices R and S are the identical.
2. Induction step. There exists y on the path from v to w such that $\text{Time}(y) = \text{Time}(K)$. Let x be the point which precedes y on the path, and z the point that follows y . Since the values of **Time** at the two ends of an arrow differ by 1, and since $\text{Time}(y)$ is minimal, we know that:

- (a) $\mathbf{Excuse}(y) = \langle x, z \rangle$ or $\mathbf{Excuse}(y) = \langle z, x \rangle$, thus $\{x, z\}$ is a fork.
- (b) The slices of x and z , say T and U , are both members of V_K .

By the induction hypothesis, R and T are in the same connected component of G_K , and so are U and S . On the other hand, $\{T, U\} \in E_K$.

□

Claim 3.3. Let $\mathbf{S} = \{S_1, S_2, \dots, S_k\}$ where $S_i \subset \mathbb{R}^d$, introduce an edge between S_i and S_j iff $S_i \cap S_j \neq \emptyset$. Assume that $\mathbf{S}$ is connected. Then $\mathbf{Size}(\cup_{i=1}^k S_i) \leq \sum_{i=1}^k \mathbf{Size}(S_i)$.

Proof. It suffices to prove that if $R_1 \cap R_2 \neq \emptyset$, then $\mathbf{Size}(R_1 \cup R_2) \leq \mathbf{Size}(R_1) + \mathbf{Size}(R_2)$. Let $a_{i,j} = \max_{v \in R_j} L_i(v)$. Then for any $w \in R_1 \cap R_2$ we have:

$$\begin{aligned} \mathbf{Size}(R_1 \cap R_2) &= \sum_i^{d+1} \max a_{i,1}, a_{i,2} = \sum_i^{d+1} a_{i,1} + a_{i,2} - \min a_{i,1}, a_{i,2} \\ &\leq \sum_i^{d+1} a_{i,1} + a_{i,2} - L_i(w) = \mathbf{Size}(R_1) + \mathbf{Size}(R_2) - \mathbf{Size}(w) \\ &= \mathbf{Size}(R_1) + \mathbf{Size}(R_2) \end{aligned}$$

□

Claim 3.4. Let $K = \langle K, v_1, \dots, v_{d+1} \rangle$ be a spanned slice of the investigation such that $K \neq \mathbf{History}(K)$. Then for some k there exist spanned slices $K_0, \dots, K_k$ and Forks $F_1, \dots, F_k$ such that:

1. $\mathbf{Excuse}_i(v_i)$ belongs to poles of $K_0, \dots, K_k$.
2. Introduce an edge between K_i and K_j iff one of the Forks $F_1, \dots, F_k$ consists of a pole of K_i and a pole of K_j . Then the graph formed from $K_0, \dots, K_k$ is connected.
3. $\sum_{i=0}^k \mathbf{Span}(K_i) + \sum_{i=1}^k \mathbf{Span}(F_i) \geq \mathbf{Span}(K) + 1$

Proof. First Notice that K doesn't contain points for which **Error** is true: they would not be connected to any other points in $\mathbf{History}(K)$ via arrows. Thus $\mathbf{Excuse}_i(v_i)$ is defined for $i = 1, \dots, d+1$. Next consider the graph $G_K = \langle V_K, E_K \rangle$ from ?? . V_K consists of those R for which $\mathbf{Time}(R) + 1 = \mathbf{Time}(K)$ and which are contained in $\mathbf{History}(K)$. Since $\{v_i, \mathbf{Excuse}_i(v_i)\}$ is an arrow $\mathbf{Excuse}_i(v_i)$ belongs to a slice R_i from V_K , for $i = 1, \dots, d+1$.

Since G_K is connected and its edges are realized by forks, there exist a minimal collection $\{K_0, \dots, K_k\}$ of slices from V_K which contains $R_1, R_2, \dots, R_{d+1}$, and which is connected in G_K . We pick a set of forks $\mathbf{F} = \{F_1, \dots, F_k\}$ which realizes a spanning tree for this collection of slices. Later we will identify edges from this spanning tree with the forks from $\mathbf{F}$.

For $i = 1, \dots, d+1$ we set $\mathbf{Excuse}_i(v_i)$ to be the i th pole of R_i . This assures (1). The set of remaining poles for $K_0, \dots, K_k$ will be equal to $\cup_{i=k}^k F_i$. This will assure (ii). We will also select poles for $F_1, \dots, F_k$ to form 'spanned forks' $F_1, \dots, F_k$ in such way that:

$$\sum_{i=0}^k \mathbf{Span}(K_i) + \sum_{i=1}^k \mathbf{Span}(F_i) = \sum_{i=1}^{d+1} L_i(\mathbf{Excuse}_i(v_i)) \geq \mathbf{Span}(K) + 1$$

This will assure (3).

Now we describe the selection of poles. Since we have to span..

□